

Question:	1	2	3	4	Total
Points:	8	10	8	10	36
Score:					

Instructors: Dr. Riadh Ksantini, Dr. Ali Alsaffar

(1) Answer the following questions.

- (a) [2 points] What are the six steps of Algorithmic Problem Solving? In which step, you represent the algorithm in pseudocode?

Solution:

- (b) [2 points] What is the difference between exact and approximate algorithms? Give an example of a problem for each case.

Solution:

- (c) [2 points] Graph problems is one of the important problem types in algorithms. Give other three names of important problem types in algorithms.

Solution:

- (d) [2 points] Consider the following algorithms.

```

1: Algorithm Q( $A[0..n-1]$ )
2:    $x \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $n-1$  do
4:     if ( $A[i] > x$ ) then
5:        $x \leftarrow A[i]$ 
6:   return  $x$ 
    
```

Is the algorithm *Every-Case* or not? Justify your answer

Solution:

- (2) [8 points] Design an in-place algorithm that searches in an array A of integers the indices of the 1st pair of elements that sum up to a given key k . The elements that sum up to k should be at different indices. If such two elements are found, then the algorithm should print their indices. Otherwise, the algorithm prints the message *There are no two elements that sum up to k* . For illustration, refer to the given example with expected output 2 and 4.

Array A	Index $\rightarrow$	0	1	2	3	4	5	6	7
$k = 11$	Element $\rightarrow$	4	-1	3	6	8	3	2	5

Solution:

```

1: Algorithm PAIRSUM( $A[0..n-1], k$ )
2:    $i \leftarrow 0$ 
    
```

```

3:  found ← false
4:  while i < n - 1 and not found do
5:      j ← i + 1
6:      while j < n and not found do
7:          if (A[i] + A[j] = k) then
8:              found ← true
9:          else
10:             j ← j + 1
11:         i ← i + 1
12:     if found then
13:         PRINT(i, j)
14:     else
15:         PRINT("There are no two elements that sum up to", k)
16:     return count

```

- (a) [1 point] What is the basic operation of your algorithm.

Solution: The basic operation is the comparison at line 7.

- (b) [1 point] What is the *Best-Case* and *Worst-Case* of the algorithm? Justify your answer.

Solution: *Best-Case:* When the sum of the first two elements are equal to k
Worst-Case: When the sum of the last two elements are equal to k or no sum of two numbers is equal to k

- (3) Consider the following algorithm:

```

1: i ← 1
2: count ← 0
3: while i * i < n do
4:     count ← count + 1
5:     i ← i + 1

```

- (a) [1 point] What is the *Basic Operation* of the algorithm?

Solution: The basic operation is the comparison $i * i < n$.

- (b) [7 points] Find the time complexity of the algorithm and find its efficiency class.

Solution: The basic operation is the comparison in $(i * i < n)$.
Let $C(n)$ be the total number of comparisons.
Since the **while** loop will test for $1 * 1, 2 * 2, \dots, i * i$ until $i^2 \geq n$, then $i * i \leq n$ (n is included for exiting the loop), or $i \leq \sqrt{n}$ and

$$C(n) = \sum_{k=1^2, 2^2, \dots, i^2} 1 = \sum_{k=1}^i 1 = i = \sqrt{n} \in \Theta(\sqrt{n})$$

- (4) Consider the following recursive algorithm.

```

1: Algorithm F( $n$ )
2:    $count \leftarrow 0$ 
3:   if ( $n > 0$ ) then
4:     for  $i \leftarrow 1$  to 3 do
5:        $F(n - 3)$ 

```

- (a) [5 points] Set up a recurrence relation for the time complexity of the algorithm $C(n)$. What is the basic operation?

Solution: The basic operation is the addition at line 5.
The time complexity recurrence $T(n)$:

$$C(n) = \begin{cases} 0 & n = 0 \\ 3C(n-1) + 3 & n > 0 \end{cases}$$

Note. In the post work, we may add +3 for the increment.

- (b) [4 points] Find the solution of $C(n)$.

Solution:

$$\begin{aligned}
CT(n) &= 3C(n-1) + 3 && \leftarrow \text{step 1} \\
&= 3[3C(n-2) + 3] + 3 \\
&= 3^2C(n-2) + 3^2 + 3 && \leftarrow \text{step 2} \\
&= 3^2[3C(n-3) + 3] + 3^2 + 3 \\
&= 3^3C(n-3) + 3^3 + 3^2 + 3 && \leftarrow \text{step 3} \\
&\vdots \\
&= 3^kC(n-k) + \sum_{i=1}^k 3^i && \leftarrow \text{step } k \\
&= 3^kC(n-k) + 3 \sum_{i=0}^{k-1} 3^i \\
&= 3^kC(n-k) + 3 \cdot \frac{3^k - 1}{2}
\end{aligned}$$

When $C(n-k) = C(0)$, $n-k=0 \implies k=n$, and

$$C(n) = 3^n \times 0 + \frac{3}{2}(3^n - 1) = \frac{3}{2}(3^n - 1).$$

- (c) [1 point] Establish the order of growth of $C(n)$.

Solution:

$$C(n) \in \Theta(3^n)$$